Masarykova univerzita
Fakulta informatiky

Documentation in Agile Development

Master's thesis

Jana Pelclová

Brno, 2014

Declaration

Hereby I declare, that this paper is my original authorial work, which I have worked out by my own. All sources, references and literature used or excerpted during elaboration of this work are properly cited and listed in complete reference to the due source.

Jana Pelclová

Advisor: doc. RNDr. Tomáš Pitner, Ph.D.

Acknowledgement

I would like to thank my advisor, doc. RNDr. Tomáš Pitner, Ph.D., for supervising this thesis and for giving me valuable advice during its creation. I would also like to thank my colleagues from Kentico Software for inspiring me to create this thesis and the company that made it possible.

Abstract

The aim of this thesis is to find a way to integrate the creation and maintenance of user documentation into the agile development environment used in Kentico Software. The first part of this thesis is dedicated to describing agile principles in general and provides details on Extreme Programming and Scrum. The next part is focused on studying the approaches to creating and structuring technical documentation, the role of Technical Writers in companies, the differences between the positions of documentation in traditional and agile environments and the types of file formats and tools used for storing source documentation files. These observations are utilized in the subsequent part of the thesis, which proposes adjustments to the environment in which the Technical Writers in the company work and offers experience gained in two real projects in which these adjustments were applied.

Keywords

technical documentation, software documentation, technical writer, agile, Scrum, project management

Contents

1 **Introduction** 4
2 **Agile methodologies** 6
  2.1 *Evolution of Agile Methodologies* 6
  2.2 *Iron Triangle* 8
  2.3 *Agile Manifesto* 9
3 **Extreme Programming** 12
  3.1 *Values* 12
  3.2 *Principles* 13
  3.3 *Selected Practices* 14
    3.3.1 Refactoring 14
    3.3.2 Pair Programming 14
    3.3.3 Coding Standards 15
4 **Scrum** 16
  4.1 *Principles and Values* 16
  4.2 *Roles* 17
    4.2.1 Product Owner 17
    4.2.2 Scrum Master 18
    4.2.3 Team 18
  4.3 *Sprint* 19
  4.4 *Events* 19
    4.4.1 Sprint Planning 19
    4.4.2 Daily Scrum 20
    4.4.3 Sprint Review 21
    4.4.4 Sprint Retrospective 21
    4.4.5 Backlog Grooming 21
  4.5 *Artifacts* 22
    4.5.1 Product Backlog 22
    4.5.2 Sprint Backlog 22
    4.5.3 Definition of Done 23
5 **Software Documentation** 25
  5.1 *Types of Software Documentation* 25
  5.2 *Technical Writers* 26
  5.3 *Quality of Technical Documentation* 28
    5.3.1 Task Orientation 29
    5.3.2 Other Characteristics 30

5.4 *Documentation in Traditional Development Environment* . . . 31
5.5 *Documentation in Scrum Development Environment* . 33
5.5.1 User Stories . . . 34
5.5.2 Role of Technical Writers . . . 35
5.5.3 Definition of Done . . . 35
5.5.4 Writing Fiction . . . 35
5.6 *Documentation File Formats* . . . 36
5.6.1 DocBook . . . 38
5.6.2 Darwinian Information Typing Architecture . . 38
5.6.3 AsciiDoc . . . 39
5.6.4 Wiki . . . 39
5.7 *Documentation Software Tools* . . . 39
5.7.1 Adobe Technical Communication Suite . . . 40
5.7.2 Help & Manual . . . 40
5.7.3 Confluence . . . 40
6 **Adaptation of Technical Writers for Agile Environment** . . 42
6.1 *Situation in Kentico* . . . 42
6.1.1 Development History . . . 43
6.1.2 Current Scrum in Kentico . . . 44
6.1.3 Technical Writers in Kentico . . . 44
6.1.4 The Documentation Process . . . 45
6.1.5 Specifics and Problems of the Documentation in Kentico . . . 46
6.2 *Used Documentation Tool* . . . 46
6.3 *Team and Collaboration* . . . 48
6.4 *Adapting the Documentation Process* . . . 49
7 **Case Study** . . . 51
7.1 *Team A* . . . 51
7.1.1 Planning the Project . . . 52
7.1.2 Backlog Grooming . . . 52
7.1.3 Sprint Planning Meetings . . . 55
7.1.4 Daily Meetings . . . 56
7.1.5 Sprint Reviews . . . 56
7.1.6 Sprint Retrospectives . . . 57
7.1.7 Backlog Grooming . . . 57
7.1.8 Summary . . . 57
7.2 *Team B* . . . 59

7.2.1 Planning the Project 60
7.2.2 Sprint Planning Meeting 60
7.2.3 Daily Meetings 61
7.2.4 Sprint Review 62
7.2.5 Sprint Retrospective 62
7.2.6 Summary 65
7.3 *Summary of the Case Study* 66
8 **Conclusion** 67
Bibliography 68

1 Introduction

Agile methodologies are being introduced in companies to provide a better environment for software development, where incremental work, frequent feedback and adaptation to change are embraced values. While agile methodologies have been analyzed in a range of publications, the creation and maintenance of software documentation is rarely covered in them. This might be caused by the second statement of Agile Manifesto: "Working software over comprehensive documentation", which is often interpreted in the way that agile development does not need documentation. While this may be true in some projects, quality technical documentation is more often than not part of successful software products.

Kentico Software has transitioned from traditional to agile development, but some parts of the implemented methodology are still unclear and imperfect. One of these imperfections is the process of creating and maintaining technical documentation of the developed functionality.

The purpose of this thesis is to analyze available options and propose adjustments to the Technical Writers' environment and processes so that their work in the agile development is facilitated. This work will serve future Technical Writers that come to the company to learn the reasons behind established processes and choices made. It can also provide valuable knowledge and experience to other Technical Writers that are supposed to work in agile environment as well as to Scrum Masters who want to better understand the Technical Writer role in their Scrum teams.

Chapter 2 begins with the description of agile methodologies and their evolution from the previously used traditional methods of software development. Introduced are agile principles and basic values. The next chapter focuses on one of agile philosophies, Extreme Programming. The values, principles and selected practices of Extreme programming are presented. The fourth chapter addresses the Scrum methodology, its principles, roles and iterative development process.

Software documentation is the topic of the fifth chapter. The creation and maintenance of software documentation in companies with the focus on user documentation is summed up in this chapter. Indi-

vidual sections are devoted to the description of the Technical Writer role, assessment of the quality of technical documentation and the differences between agile and traditional development environments with regards to documentation. Selected formats and tools for the creation and maintenance of software documentation are highlighted in this chapter as well.

The purpose of the sixth chapter is to analyze the current situation of project management in Kentico as well as the problems of creating and managing user documentation in the company. Subsequently, actions to be taken to facilitate the integration of user documentation creation into the Scrum process are offered along with the suggestions for adjusting the documentation process in the company.

The next to last chapter provides insight into real examples of putting the theory into practice. Two separate projects in two teams are presented from the Technical Writer's point of view to show, which aspects of the offered solution required further modifications.

The thesis is summarized and concluded in the last chapter.

2 Agile methodologies

This chapter presents the predecessor of agile methodologies and circumstances that led to a new philosophy of software development. Main principles of agile methodologies are explained subsequently.

2.1 Evolution of Agile Methodologies

Before agile methodologies of software development can be properly understood, it is important to be familiar with their predecessors and historical background.

The agile methodologies evolved from traditional plan-driven development methods. The principle of the traditional methodologies is that the development process is sequential and divided into phases. Only after one phase is finished, can the work on the next phase begin. These methodologies suppose that if a project is analyzed and well planned at the beginning, the better will be its execution [19]. One of the most known traditional development methodologies is the Waterfall process. The sequential scheme of Waterfall can be seen on figure 2.1.

The principles and phases of the Waterfall process are logical. At first, the purpose of the project is specified along with the requirements of the final product. Then the product is designed and planned. Subsequently, the product is implemented according to the plan and only after that it is tested. Finally, the product is deployed to the target environment or just released. Each phase depends on the preceding phase and logically builds on it further. This approach assumes that all phases are distinct and strictly separated from each other. Another prerequisite for this approach to work is that the product to be developed is well defined, predictable and the requirements are not expected to significantly change during the development [19].

The problem is that most software projects do not work that way. Software projects are difficult to plan completely at the beginning and they often need to be modified during the development cycle. For example, the contractors of the project may not know exactly which features they would need in the product. They might need some visual prototype at first to refine their perspective. It is also

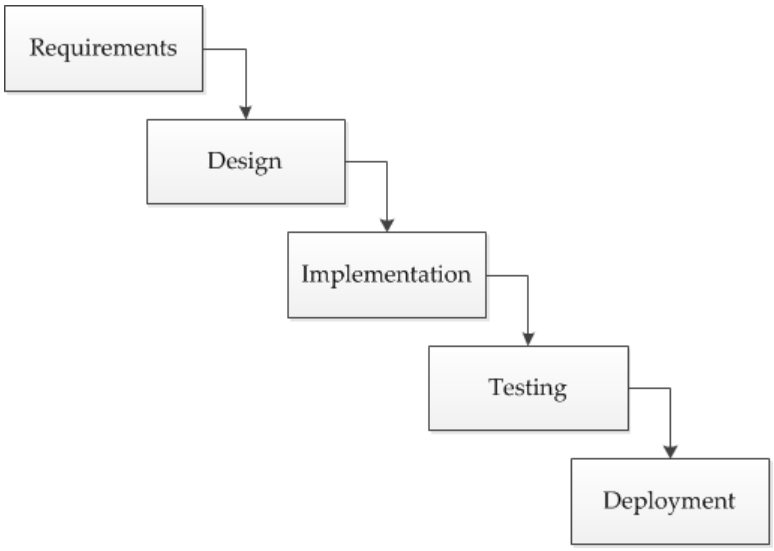


Figure 2.1: The Waterfall process

possible that they adjust the requirements during the course of the project due to changes in technologies or as a response to competitor products on the market. In the Waterfall process, these changes in the plan are costly and often lead to increased expenses for the whole project or to postponing the release deadline. Moreover, estimating the duration of a long project at its beginning is rarely accurate and leads to delays in individual phases of the cycle [10].

Therefore, the Waterfall methodology is suitable mainly for projects with clearly defined requirements with a low probability of changes during the development cycle. Projects, where it is unlikely that new requirements will arose and where there is enough time and resources to profoundly complete each phase of the cycle.

On the contrary, the agile methodologies are adapted to the current demand of software development, which is prompt response to changes [13]. The technologies are quickly evolving and getting obsolete, and the companies need to keep pace with the competition on the market. Developed functionality is often consulted with the contractors or target users and adjusted according to their needs. Agile builds on incremental work, where the changes are welcome and easily incorporated into the product. This approach also assumes that

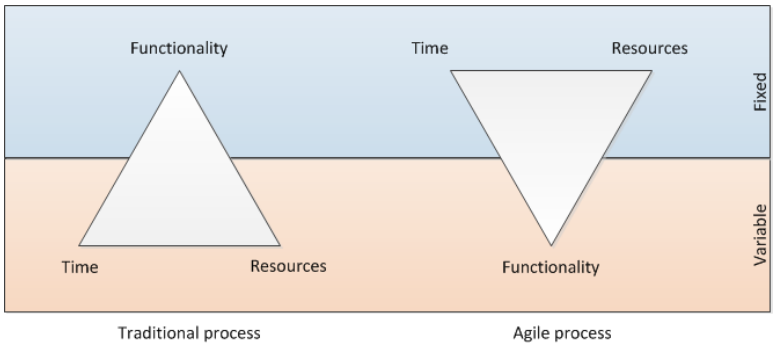


Figure 2.2: The triple constraints in traditional and agile processes (source: [17])

creating increments of the product that are potentially releasable can benefit the companies and bring them an advantage on the market [8].

2.2 Iron Triangle

The iron triangle principle illustrates the differences between the factors that influence the traditional and agile methods. It shows, which conditions are set and which are variable, as can be seen on figure 2.2.

In traditional development, the unchanging factor for the project is the required functionality for which the time and costs are estimated and planned. The aim is to deliver the required functionality within the planned time scope while not exceeding the assumed expenses. Nevertheless, due to the risk factors accompanying this type of project management and stress the people participating on the project experience at the end of the expected time scope, there is a probability that the costs will be exceeded and release dates delayed. There is also a risk, that the customer will not receive any working functionality on the account of depleting the budget before finishing the project.

On the other hand, the agile approach works with the provided resources and assigned time and tries to deliver as much functionality as possible. This way, the expected time and resources will not

be exceeded, but the customer may not receive a product with all the required functionality. Whole parts of the functionality are implemented incrementally according to the customer's priorities and current conditions, but it is ensured that the customer is presented with a working product.

2.3 Agile Manifesto

The upswing of the agile era came when the Agile Manifesto was written. In 2001, seventeen practitioners of various methodologies gathered on a summit and agreed upon 4 main values of better software development. Subsequently, they added 12 principles that further explain the ideas behind agile methodologies [1].

Agile Manifesto defines the following values [5]:

- Individuals and interactions over processes and tools
- Working software over comprehensive documentation
- Customer collaboration over contract negotiation
- Responding to change over following a plan

"That is, while there is value in the items on the right, we value the items on the left more."

The first value implies that the people and interactions among them are essential in software development teams. The communication in the team should be easy and quick and preferably face-to-face to achieve the best performance of the team. This does not mean that processes and tools are not important in agile development, only that effective communication is valued more.

The second value puts emphasis on working software, which is delivered in small increments in defined intervals. It also promotes testing of the software for ensuring its quality and operability. This principle is often misperceived, as it is believed that the software does not need any documentation at all. Such assumption may not be favorable in projects where certain types of documentation, for example the user manuals, are needed.

The next value promotes continuous collaboration with the customer. It is not enough to conclude a contract, engaging the customer into the software development process is essential for creating valuable software [25]. The customers should provide feedback regularly through the development, as only they know which functionality will bring them the highest value and in which order.

The last value represents the essence of agile processes, which is the response to changes. Software development is a dynamic environment where constant changes need to be incorporated into the existing product. Thus, planning a complex software project in advance is ineffective and sometimes even impossible. The principles behind the Agile Manifesto are the following [5]:

- Our highest priority is to satisfy the customer through early and continuous delivery of valuable software.
- Welcome changing requirements, even late in development. Agile processes harness change for the customer's competitive advantage.
- Deliver working software frequently, from a couple of weeks to a couple of months, with a preference to the shorter timescale.
- Business people and developers must work together daily throughout the project.
- Build projects around motivated individuals. Give them the environment and support they need, and trust them to get the job done.
- The most efficient and effective method of conveying information to and within a development team is face-to-face conversation.
- Working software is the primary measure of progress.
- Agile processes promote sustainable development. The sponsors, developers, and users should be able to maintain a constant pace indefinitely.

- Continuous attention to technical excellence and good design enhances agility.
- Simplicity – the art of maximizing the amount of work not done – is essential.
- The best architectures, requirements, and designs emerge from self-organizing teams.
- At regular intervals, the team reflects on how to become more effective, then tunes and adjusts its behavior accordingly.

3 Extreme Programming

Extreme Programming (XP) is a collection of principles, values and practices forming an agile approach to software development. It shares many ideas with the general agile environment, such as developing in short cycles, incremental planning, quick and frequent response to customer feedback, but focuses more on the actual development practices. XP is designed to produce high-quality software in an environment of rapidly changing requirements.

Extreme Programming has been selected as a representative of agile approaches to software development. The reason for including it in this thesis is that its values and principles can be applied in any software development environment. Furthermore, some practices of XP (described in section 3.3) have already been tried out in Kentico.

Unless otherwise stated, this chapter draws from [4].

3.1 Values

The four main values of XP are:

- Communication
- Simplicity
- Feedback
- Courage

Communication is critical in software development. Developers, testers, managers, customers and other roles need to communicate among themselves clearly and easily to achieve a common goal. Extreme Programming employs techniques that keep the right communication flowing.

Simplicity means that effort should not be spent of work that is not currently needed. The developed features should be as simple as they can to satisfy the requirements.

The more feedback about the system the better. Programmers and testers write unit test and functional test to see if everything works

correctly. The programmers estimate the work to be done and track their progress so that the managers and customers know when they are likely to finish. The product is developed incrementally to collect the customers' feedback on functional parts of the product.

It takes a great deal of courage to throw away a day's work of code, but even this may in certain cases be beneficial. Also, when fixing one flaw breaks half of the tests, it is not easy to keep high spirits, and courage helps overcome such situations. XP supports courageous thinking in combination with the three previous values.

Moreover, XP cherishes a fifth value – respect. The whole philosophy cannot be successful when the members of the team do not respect each other and do not believe in their work.

3.2 Principles

While the values presented in the previous section may be vague and different people may understand them differently, the principles are more concrete. The fundamental principles of XP are:

- Rapid feedback
- Assume simplicity
- Incremental change
- Embracing change
- Quality work

Not only some feedback, but rapid feedback and quick reaction to it are fundamental in XP. When feedback is received, it has to be interpreted and incorporated back in to the system as soon as possible.

The best way to solve the most problems is using the simplest way possible. XP says that it is better to spend only necessary amount of effort for completing tasks and that complexity can be added anytime in the future.

XP states that making big changes at once is not effective. It is much more convenient to do series of small changes. Even XP principles have to be adopted gradually.

Embracing change means, that the development does not try to do everything perfect from the beginning but understands and encourages changes.

Only when people do an excellent work can they enjoy it and be proud of its results, otherwise the whole project is prone to a failure.

3.3 Selected Practices

XP introduces several practices which explain how this philosophy can work together as a whole. Only selected practices are described in this chapter.

3.3.1 Refactoring

With every work the programmers do on already existing programs, the programmers think whether it is possible to change the existing program so that it is easier to add the new functionality. After the new functionality has been added, the programmers also try to make the program simpler while preserving all the functionality and passing all tests. This approach sometimes means to do more work than necessary, but can save time later when adding other code.

3.3.2 Pair Programming

The concept of pair programming is having two programmers work on the same issue on one computer. The programmers act in two roles. The person at the keyboard is the driver and the other navigator.

The driver is the one who writes the code as cleanly as possible, while the navigator thinks about how the code fits into the rest of the program or which test scenarios have to be considered [23].

Pair programming produces high quality code, helps share knowledge throughout the team and enables new members of the team to integrate quickly.

3.3.3 Coding Standards

Especially when pair programming and refactoring is practiced, the code needs to be written using the same standards across the company to ensure consistency and easy communication through the code.

4 Scrum

Scrum was chosen as a representative of agile methodologies because this particular methodology is currently being used in Kentico. For the purpose of this thesis, main principles, roles, events and artifacts of Scrum are described in this chapter.

4.1 Principles and Values

Scrum is a framework for developing complex products. It is based on agile principles and the Agile Manifesto and enriches these principles of its own values, rules and practices. It is not a standardized process with clearly defined steps to follow, but rather a framework for organizing and managing work [19]. Scrum is a foundation, on which each organization can implement various practices and approaches. Scrum is [21]:

- Lightweight
- Simple to use
- Extremely difficult to understand

Understanding the basic principles of Scrum is not difficult and anyone can learn them quickly from sources publicly available on the Internet. The implementation of the methodology in practice though, is a long-term process that requires a different way of thinking about the management of the product development.

Scrum is best suited for environments, where the development of a product requires variability and adaptation to changes. Furthermore, it helps receive feedback quickly and frequently to confirm the direction of the development. It also assumes that the process of creating a product is too complex to have a complete and clear definition.

Scrum is based on three principles:

- Visibility
- Inspection

- Adaptation

Visibility means that those aspects of the process that affect the resulting product must be visible and transparent to all participants of the process. For example, the definition of a planned functionality must be comprehensible to those who plan the functionality as well as to those who will implement it. The second principle implies that the aspects of the process need to be inspected regularly, so that any variances in the process can be detected in a timely manner. If the inspection shows that any aspects are out of the boundaries of the Scrum framework and therefore would negatively influence the resulting product, the process is adjusted straight away [22].

4.2 Roles

To understand the Scrum process thoroughly, it is essential to be familiar with the roles that occur in the process. There are only three main roles in Scrum:

- Product Owner
- Scrum Master
- Team

Together, they are called the Scrum Team. All management responsibilities in the project lie among these three roles. People in these roles are committed to the project and therefore, only they have the authority to do what is necessary for its success. Anyone else outside of these roles cannot interfere in the process. Scrum strictly distinguishes between these two groups and makes sure that it is always clear who performs and who only stands by. This model is designed to optimize flexibility, creativity and productivity [21].

4.2.1 Product Owner

The Product Owner represents the customers and decides which functionality will be developed and in what order. The Product Owner responsibility is to ensure the return of the investments in the project

and to maximize the value of the developed functionality. This is achieved by managing the Product Backlog (see section 4.5.1). The Product Owner ensures that it is visible and its items are clearly defined and prioritized in a list.

The Product Owner can represent external stakeholders or inner interests of the company, but it is always only one person.

4.2.2 Scrum Master

The Scrum Master is responsible for implementing and following the Scrum methodology itself. The person in this role helps everyone involved in the project understand the values and principles of Scrum, adhere to the principles and adjust the processes and practices to best suit their environment. The Scrum Master acts as a coach, creating favorable conditions for work, removing impediments which distract the Team from being productive, helping the Team develop their skills and knowledge and supporting the Team in self-organization. Furthermore, the Scrum Master convenes and facilitates Scrum meetings (see following sections).

This role differs greatly from the traditional project manager role. While project managers have authority over the development progress and the organization of the team, the Scrum Master is rather a facilitator and coordinator of the methodology.

4.2.3 Team

The Team is a cross-functional group of people with skills needed to develop the desired product. There can be programmers, testers, UI designers, architects, and other specialized professions, but it is always the whole Team who is responsible for the delivered product. The Team is self-organizing and no one tells them how to turn items in the Product Backlog into potentially releasable increments of functionality. They alone decide on how to best achieve the goal of each Sprint (see following section).

The size of the Team is large enough to be able to perform the required work and have all the skills needed. It should also be small enough so as not to unnecessarily increase the amount of interactions between the individual members. Therefore, the optimal size of one

Team is between 5 and 9 members [19].

4.3 Sprint

Sprint is a fundamental process of Scrum. It is a short time period, lasting one to four weeks, during which the Team develops a potentially releasable product increment. The duration of the Sprint is always fixed and depends on the company's environment and needs.

Each Sprint begins with a Sprint Planning meeting, ends with a Sprint Review meeting and is concluded with a Sprint Retrospective meeting. In addition, a short Daily Scrum meeting is held every day of the Sprint.

4.4 Events

Events are defined in Scrum to support regularity and to decrease the number of other meetings not defined in Scrum, so that the Team can focus on the actual work. All events have predefined time span, which is determined by the type of the meeting and the chosen length of Sprints. They also have a designated place in the incremental development process, as can be seen on figure 4.1.

4.4.1 Sprint Planning

The Sprint Planning Meeting is held at the beginning of each Sprint. Its purpose is for the Scrum Team to gather and plan the work to be performed in the particular Sprint. This meeting is divided into two parts.

The first part of the meeting answers the question: "What will be delivered in the Increment resulting from the upcoming Sprint?" [21]. The Product Owner presents and explains the requirements on the functionality with the highest priority. The Team then discusses the requirements and tries to assess the maximum number of requirements they can complete in the upcoming Sprint based on the amount of already completed functionality in previous Sprints.

The second part of the meeting answers the question: "How will the work needed to deliver the Increment be achieved?" [21]. The

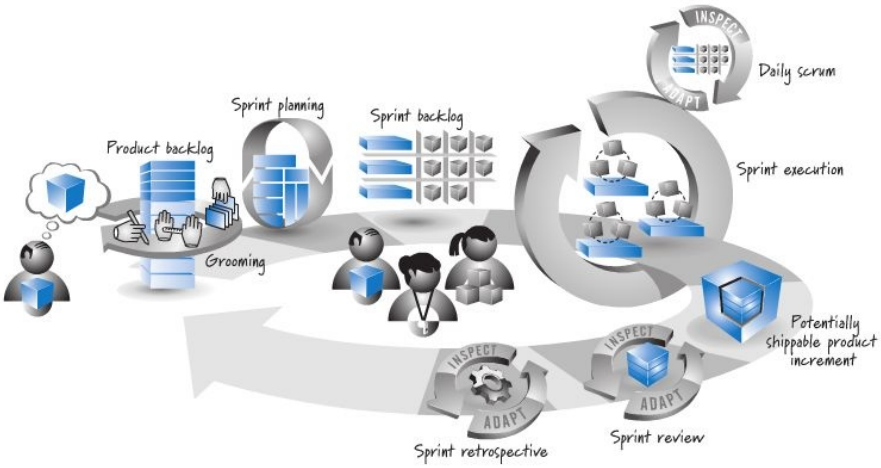


Figure 4.1: The Scrum framework [19]

Team outlines the course of the development and decomposes the required functionality into smaller sub-tasks needing one day of work at the most.

The greatest difference from the traditional planning of projects is that the Team decides on the amount of functionality planned for the Sprint, not a manager or Product Owner. The Team commits to complete the plan within the Sprint and self-organizes to divide the work among its members.

4.4.2 Daily Scrum

Every day of the Sprint, the Team together with the Scrum Master hold a Daily Scrum meeting to synchronize their activities. The Daily Scrum takes the maximum of 15 minutes and happens at the same place and time every day. During this meeting, each member of the Team answers three questions:

- What have you accomplished since the last meeting?
- What will you do before the next meeting?
- What impediments are in your way?

An essential concept here is that the members of the team do not report to the Scrum Master or Product Owner but instead, they report to each other. The Scrum Master only ensures that the meeting is held and that it does not exceed 15 minutes. The Product Owner does not participate in Daily Scrums. The purpose of this meeting is to plan the work of the Team for the next 24 hours [21].

4.4.3 Sprint Review

The Sprint Review meeting is planned at the end of each Sprint and its purpose is to review and present the work done during the Sprint. The Team demonstrates completed functionality to the Product Owner and the stakeholders. The Product Owner provides feedback on the completed functionality and carries any further requirements over to the Product Backlog. Based on what has been done and what has not been done in the Sprint, the Product Owner together with the Team contemplates on the following course of work.

4.4.4 Sprint Retrospective

To conclude each Sprint, the Sprint Retrospective meeting is planned after the Sprint Review and before the next Sprint Planning meetings. While the Sprint Review meeting is designed to review and adapt the product, the Sprint Retrospective meeting provides an opportunity to inspect and adapt the process [19]. The Team and, sometimes also the Product Owner, with the help of the Scrum Master reflect on what went well in the past Sprint regarding the communication, relationships, processes and tools. They discuss problems that arose during the Sprint and try to find ways to improve their process within the Scrum framework. The Scrum Master encourages the Team to find solutions to identified issues so that the development process in more effective and enjoyable in the next Sprint.

4.4.5 Backlog Grooming

Backlog grooming is an activity in which the Team and the Product Owner collaborate on refining the items in the Product Backlog (see the next section). Together, they add more detail to the requirements

in the list, order the items and decompose requirements that are too large and would not fit into one Sprint. The Team also estimates the amount of work needed to complete several highest requirements in the list.

It is a supplementary meeting, which is not mandatory in the Scrum process, but it can help clarify complex tasks and provide better input for the Sprint Planning meeting.

4.5 Artifacts

Scrum uses artifacts to keep track of tangible things in the project. The artifacts represent work and value and are designed to provide visibility in the whole Scrum process. When the work is visible, it can be often inspected and adapted as needed.

4.5.1 Product Backlog

The Product Backlog is an artifact in the form of a list of all requirements for the developed product. The list is never complete and any items can be adjusted and new items added anytime based on feedback from customers, situation on the market and other factors. The only person responsible for the content of the Product Backlog and its relevance is the Product Owner. The Product Owner also orders the items in the list according to the value or other priorities.

An important property of the Product Backlog is that it must be visible and available for inspection to everyone involved in the project.

4.5.2 Sprint Backlog

The Sprint Backlog is a set of items from the Product Backlog selected for the Sprint. The Team selects the items in the first part of the Sprint Planning meeting and then creates and defines sub-task, which the members of the Team will have to carry out to complete the items, in the second part of the meeting [22]. The tasks should be small enough so that they do not take more than one day of work of one person.

Only the Team manages the Sprint Backlog and they report their

progress on individual tasks from the list on Daily Scrum meetings, on which they also select new tasks to work on next.

4.5.3 Definition of Done

The Team creates an increment of the product every Sprint. The increment is thought of as potentially shippable, because the Product Owner might have further business plans with it. The Product Owner may decide to release it immediately or show it to the customers to gain feedback. Therefore, it should be clear what completed functionality means and when is the work "done".

The Team defines its own Definition of done based on their environment, the developed product, used technologies and conditions set by the Product Owner. The Team can adjust the definition when necessary.

The following is an example of a Definition-of-done list for work done during a Sprint [19]:

- Design reviewed
- Code completed
  - Code refactored
  - Code in standard format
  - Code is commented
  - Code checked in
  - Code inspected
- End-user documentation updated
- Tested
  - Unit tested
  - Integration tested
  - Regression tested
  - Platform tested
  - Language tested

- Zero known defects
- Acceptance tested
- Live on production servers

5 Software Documentation

This chapter provides an overview of software documentation with focus on end-user documentation. It also familiarizes the reader with the role of Technical Writers in a company. Subsequently, this chapter elaborates on differences of software documentation specifics in traditional and agile development environments. And finally, several formats and tools used for producing the documentation are discussed as well.

5.1 Types of Software Documentation

Software documentation represents all written documents and materials that accompany computer software. According to [24], software documentation can be divided into two categories:

- Process documentation
- Product documentation

Process documentation records the progress of the development cycle. Into this category fall materials that are created for the purpose of keeping track of the project, passing and storing information about the project as well as plans, estimates and other information that helps make the whole process visible. Visible processes enable more effective management of projects. Types of documents from this category include project plans, test schedules, reports, standards, meeting notes or business correspondence.

Product documentation focuses on the developed software product. These materials offer descriptions and information on the product and provide instructions on how to perform various tasks with the product. Product documentation includes:

- User documentation
- System documentation

User documentation are materials which are mainly prepared for end-users of the product and system administrators. They advise

the users on working with the product and provide instructions on how to perform various tasks. These materials also help the system administrators install and maintain the system. User documentation includes tutorials, user guides, troubleshooting manuals, installation and reference manuals.

System documentation represents documents that describe the system itself and its parts. System documentation includes requirements specifications, design specifications, architecture description, descriptions of functionality and interfaces, program source code listings and validation documents.

The composition of documents produced for software projects depends on various aspects. The required documents may be determined by the type of the project and its environment, contract with the customer, company politics and the methodology used for managing the software development. Each software project may need different documents, for example, a complex photo editing software may utilize a comprehensive end-user documentation, while an internal tool developed for the needs of the company may require only a system requirements list. The contractor may explicitly specify that certain documents, for example the release notes, must be created for the product. The company politics also influence the required types of documents by, for example, adhering to ISO standards, which formulate additional demands on documentation materials.

Traditional development methodologies rely heavily on process documentation and descriptions and specifications of the product. Agile methodologies, on the contrary, try to minimize the process documentation and emphasize the significance of oral face-to-face communication.

5.2 Technical Writers

The United States Department of Labor [7] defines the Technical Writer occupation as follows: "Technical writers, also called technical communicators, produce instruction manuals and other supporting documents to communicate complex and technical information more easily. They also develop, gather, and disseminate technical information among customers, designers, and manufacturers."

Generally, Technical Writers are professional authors of documentation. Their responsibility is to explain complex concepts and technical information in a comprehensible way that is appropriate for the target audience. Technical Writers can be found in various fields of study, including medicine, physics, chemistry, engineering, software development and others. In each field, they may produce different documents, but their main goal remains the same – to study the subject and compose the gained information in documents.

The main skills expected of Technical Writers are excellent expertise in the target language and broad knowledge in the field in which they work. However, the Technical Writers need not necessarily be professionals in the technical fields. It is only essential that they are familiar with the required technologies, technical expressions and the environment to comprehend the matter. In software development, Technical Writers do not need to be programmers, but they must be able to understand written code and explain what its parts do. They can be also required to modify sections of code written by programmers to produce code examples.

Excellent communication skills are also indispensable in Technical Writers, because they gather information either by studying the subject or by talking to Subject Matter Experts (experts in the particular fields, commonly abbreviated as SMEs). In software development, Technical Writers experiment with the documented software and interview the programmers and testers about the functionalities of the software. An emphasis is put on the Technical Writers' soft skills, which are fundamental for keeping good relationships with Subject Matter Experts, gaining information easily, working in teams, planning and organizing the working time effectively.

Software companies either employ their own Technical Writers or outsource the creation of the documentation. This depends on the character of the software being developed and its demands on the documentation. Complex products with long-term development, which require detailed user guides and manuals benefit from having Technical Writers employed full-time. This way, the documentation can be created more quickly and precisely with the collaboration of Subject Matter Experts. Outsourcing the documentation work is suitable for products developed using the Waterfall methodology, where the documentation is created at the end of the project. The rea-

son why the Subject Matter Experts do not write the documentation themselves is that they are too submerged in their field of expertise. They may not see the matter from a broader perspective and thus write the information in the best way to suit their audience [6].

From the technical documentation types described in the previous section, Technical Writers mostly create documents form the user documentation category. They produce user guides, tutorials, installation and troubleshooting manuals and other help for the users. However, they can also collaborate on the creation of system documentation with SMEs. This practice is useful from various reasons. Firstly, the Technical Writers can review the language in which the requirement specifications and other materials are written, ensure that everyone understands them in the same way and that the text is written clearly and concisely. Secondly, while writing or reviewing the code comments, the Technical Writers act as QA engineers. When testing what the code actually does, they may find bugs in the system and thus supplement the QA engineers' work.

Moreover, Technical Writers may assist with the creation of Process documentation by clarifying the language and ensuring that the text is understandable for all participating parties.

5.3 Quality of Technical Documentation

People search for information in technical documents when they need to know something. They might want to learn about the system, solve a problem, perform a complicated task or configure the behavior of the system. People look for the documentation only when they need to, and they often have limited time and patience for finding the required information. Unfortunately, when people have bad experience with the technical documentation, they generally begin to regard the documentation as unhelpful and unusable. If they do not manage to find particular information or, on the other hand, they find a topic about their problem which does not help them with their task, they lose interest in the documents.

Therefore, the technical information needs to have certain qualitative characteristics, so that it can benefit and not frustrate the users of the software. According to [12], quality technical information can

be characterized by three groups of values with the following characteristics:

- Easy to use
  - Task orientation
  - Accuracy
  - Completeness
- Easy to understand
  - Clarity
  - Concreteness
  - Style

  Easy to find
  - Organization
  - Retrievability
  - Visual effectiveness

Writing task-oriented technical instructions is essential in agile environment. Therefore, a whole sub-chapter is dedicated to this characteristic, while other characteristics are explained only briefly.

5.3.1 Task Orientation

Task-oriented writing focuses on aiding users with their tasks. It is often important to explain to the users how a product works and how it is structured, but this information rarely helps the users directly with accomplishing their goals. Feature oriented documentation approach, which encompasses writing descriptions of the product, explaining the product's features and elements of the user interface, corresponds with the traditional development methodologies. In the Waterfall model, the documentation is produced at the end of the project, separately from the development process. It is not created in a close collaboration with the product creators and thus is focused more on what can be done with the product instead on what the users actually need to do with the product.

Instead, it is more useful to provide the users with practical instructions on how to perform individual tasks with the product. To determine the most important tasks the users will need to perform, the Technical Writers need to be familiar with the target users' environment and conditions. Tasks with clearly written instructions and steps help users quickly accomplish what they need. This approach is favored by agile methodologies, as it corresponds with task-oriented recording of requirements in the form of user stories.

The next level of task-oriented documentation writing is workflow-oriented documentation, as suggested by Leah Guren in [11]. This approach not only emphasizes structuring the information in tasks but also promotes logically connecting the tasks into workflows. Workflows can be understood as sequences of tasks that need to be performed chronologically to achieve a certain goal or as circumstances in which the users perform the tasks. This approach, however, requires even broader knowledge of the audience and their needs, and deeper understanding of the documented system.

5.3.2 Other Characteristics

Accuracy means that the technical information is correct and free of errors. Users of the software rely on the accuracy of the documentation and may lose their trust in it if they encounter information that is incorrect. Therefore it is a good practice to have SMEs check the documentation for technical accurateness and to closely follow the development of the product to keep the documentation up-to-date.

The technical documentation must contain all the necessary information on the subject. It is, however, important to determine the ideal amount of information which should be included in individual topics. Too little information may frustrate the users, when they do not find the information they need. Too much information though, may overwhelm the users and make the particularly sought information impossible to find.

Clear information enables the users to understand the written text the first time they read it. The language in which the information is written is simplified and does not contain complex structures and vague descriptions.

Concreteness means that abstract concepts and theoretical aspects

of the product are brought closer to the user by using appropriate examples, analogies and illustrations.

Style defines writing conventions that the writers in the company should adhere to. Such standards cover the tone in which the text is written, grammar and spelling and other editorial choices. They enable Technical Writers to write with the regards to the corporate identity and help the users receive the information they expect.

Organizing the information in a logical and predictable way is another important trait of technical documentation. It applies to organizing the topics into structures as well as arranging the information inside individual topics. The information should be organized consistently to allow for easy navigation.

Retrievability means that information in technical documentation is easy to find. It is important to ensure that the users do not get lost while searching for specific information and that they find it quickly and by using various approaches.

Visual effectiveness implies that the visual elements in the documentation complement the text and highlight important information. Illustrations clarify significant concepts that would be difficult to comprehend from text and provide examples of usage of the product. Visual effectiveness also influences all other characteristics of quality technical information and thus need to be in balance with them.

5.4 Documentation in Traditional Development Environment

In traditional Waterfall approach to software development, documentation is created mainly at the beginning and end of the project, as illustrated on figure 5.1. At the beginning, requirement specifications are created in the form of documents describing the project to be developed, its features, attributes and behavior. Architecture and design documents define visual and construction principles to be used in the product. Project plan states what and when will be finished and test plan what, when and how will be tested.

The disadvantage of creating heavy project documentation at the beginning is that it is difficult to capture all properties of the product

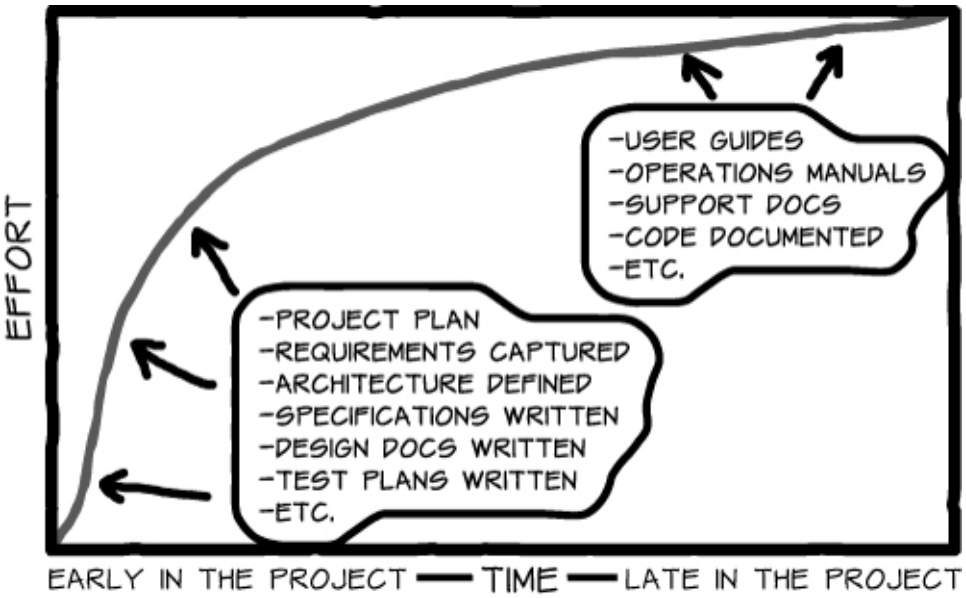


Figure 5.1: Documenting heavily at the beginning and end of the project (source [15])

before its implementation begins. Consequently, these documents often become outdated as soon as they are created [15].

Technical Writers in traditional methodologies create user documentation at the end of the project, after the product has been developed. This approach brings advantages as well as disadvantages. The advantage is that the user documentation can be provided as a service, either by the company's separate documentation team or by outsourcing. When the product is completed, the Technical Writers can explore it with all its provided features, finished user interface and unchanging environment.

The problem is, that often this is not enough and the Technical Writers also need to obtain information from the Subject Matter Experts (SMEs), who may be hard to reach after the development phase is finished or may not remember important details from the product implementation. Another difficulty is that the time needed to create the user documentation further extends the overall time of the project and thus postpones the product release. This fact also puts stress on the Technical Writers to finish the documentation in the

scheduled time.

5.5 Documentation in Scrum Development Environment

The Agile manifesto [5] emphasizes working software over comprehensive documentation. However, this does not mean that products developed under agile methodologies do not need documentation at all [20]. Software documentation is still indispensable part of projects, though agile methodologies change the schedule when such documents are created. Besides, agile principles promote face-to-face communication and, consequently, minimize the amount of process documentation produced.

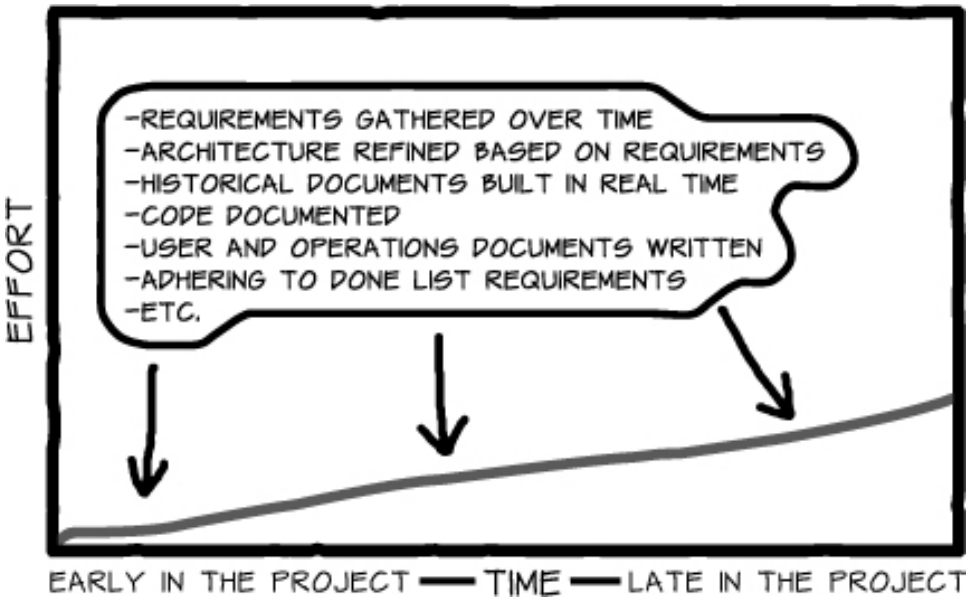


Figure 5.2: Documenting throughout the whole project (source [15])

The opposite to creating documentation at the beginning and end of the project (as described in the previous section) is creating documentation throughout the whole project, as illustrated on figure 5.2.

Only a necessary amount of requirements is specified at the beginning of the project and the requirements are further refined and

added during the project by the Product Owner. The requirements are not compiled into an extensive continuous document, but are recorded in the form of user stories.

User documentation is not created all at once at the end of the project, after the product has been implemented. It is created continuously within individual Sprints and updated as the functionality of the product develops. This approach bears its own challenges and benefits, the most significant of which are described in the following sections.

5.5.1 User Stories

User stories are short texts capturing the interaction of a user and the system. They are used to represent the work that is to be done and focus on value the user gains from the feature, rather than the technical implementation of the feature. User stories are written on sticky cards to ensure that the information included in them is kept to minimum. A typical user story has three parts [16]:

- Title
- Description
- Acceptance criteria

The title is only used to clearly differentiate the story from other stories. The description states what a particular user role wants to achieve with the product. It is written down in the following format:

As a [user role] I want to [goal] so I can [reason].

Acceptance criteria define when the user story is satisfied. They can be written as test cases or as a description of finished story.

Having the product requirements defined through user stories helps Technical Writers start writing the user documentation as soon as a Sprint is started. The user role specified in a user story states who is the documentation created for. The acceptance criteria in the form of test cases provide an initial draft of task based user documentation and also suggests which tasks will be important for the users to perform.

5.5.2 Role of Technical Writers

The position of Technical Writers shifts from an independent service to a part of the process. In agile development, documentation is included in the work done during Sprints. Therefore, it is best that the Technical Writers become members of the Team in Scrum terminology and physically move to the same office or place with the Team. Such practice encourages communication between the developers, QA engineers and the Technical Writers. It also enables the Technical Writers to participate on Scrum meetings and thus be more involved in the creation of the product, which can be beneficial for both sides. The Technical Writers closely represent the users of the system and can provide valuable feedback on the developed functionality while gaining technically accurate knowledge about the functionality directly from the members of the Team.

5.5.3 Definition of Done

As the documentation is part of the product and increments of the product are delivered every Sprint, it should be stated that completed work in a Sprint includes also completed documentation of the implemented functionality. Consequently, created or updated user documentation should be added to the Team's Definition of done to ensure, that it receives proportionally similar amount of attention as development and QA tasks. An example of a definition of done containing updated user documentation and supplied code comments can be found in section 4.5.3.

5.5.4 Writing Fiction

Due to the fact that the documentation is created in Sprints, its creation cannot wait until the functionality planned for the Sprint is developed. There is usually not enough time for this left at the end of the Sprint. Therefore, it is important that Technical Writers learn to write user documentation about features that are not developed yet. They can write drafts of documentation in the form of tasks based on acceptance criteria in user stories, prototypes and wireframes and sketches of the user interface [3].

It is certain that parts of the written fiction will have to be rewritten once the features are finished, but this way the Technical Writers can ensure that the documentation is created in time. Reviewing and rewriting parts of the documentation also make it more refined, accurate and helpful.

5.6 Documentation File Formats

When choosing the format for storing source files for software documentation, several options are available. They can be basically organized in three groups, which are summed up in table 5.1. These formats support single source publishing, where the data is stored using one common format and can be exported to various output formats using help authoring software tools (presented in the next chapter).

Markup languages are systems, where additional information, in the form of tags and commands, is used in text to define the structure of a document and elements within it. Their advantage is that the content is completely separated from the presentation form and therefore the output formats can be highly customized for individual needs. The problem is that, even though the tags and commands are usually human-readable, they make the source text difficult to decode. Working with these formats also requires some initial training and their application should be considered with regard to the writers' skills.

Lightweight markup languages are markup languages with a much simpler syntax. They are designed for documents, where users with various technical skills are expected to work with the source text as well as the final output. These formats are easy to learn but provide fewer formatting and customization options.

Proprietary systems use their own specialized formats of source files. This fact normally does not cause problems when working with the tool, as the tool can utilize the specialized format effectively and produce various outputs. However, migrations to other formats and tools are problematic in this case.

The following sections describe selected representatives of listed format groups.

Option	**Representatives**	**Advantages**	**Disadvantages**	**Typical usage**
Markup languages	DocBook DITA HTMLBook	Rich customization options. Advanced structuring options. Extensive output customization options.	Different content editing style of work from WYSIWYG editing.	Where a number of output (including customized) formats is required.
Lightweight markup languages	Wiki AsciiDoc Markdown	Easy to learn for technical users.	Limited customization options. Formatting elements instead of semantics. Absence of advanced structuring. Problematic conversion to other source formats.	Where less experienced users are required to edit the documents. Where advanced output customizations are not needed.
Proprietary systems	Adobe RoboHelp MadCap Flare Help & Manual	Easy to learn for anyone.	Limited customization options. Limited input formats. Nontrivial output customizations.	Where single supported input format is not a problem. Where advanced output customizations are not needed.

Table 5.1: Comparison of documentation source format options (inspired by [26])

5.6.1 DocBook

DocBook is a semantic markup language suited for writing technical documentation. This format has been developed since 1991 with the current version of 5.1 being maintained by the DocBook Technical Committee of Organization for the Advancement of Structured Information Standards (OASIS). This source format can be easily converted to several output formats, including HTML, PDF and RTF, using publicly available tools. DocBook provides almost 400 element tags, which are divided into three categories: structural, block-level, and inline [14].

Structural tags specify structural characteristics of the text, such as chapters or articles. Block-level tags represent elements in the text, such as paragraphs or lists. Inline tags wrap parts of the text to apply typographical formatting on them.

5.6.2 Darwinian Information Typing Architecture

Darwinian Information Typing Architecture (DITA) is a XML standard for authoring and producing technical documentation maintained by the DITA Technical Committee of OASIS. The name is based on Charles Darwin's theory of evolutionary adaptation which illustrates the principle of specialization and inheritance used by this standard. DITA specializes on topic-based authoring. Topics are standalone units of information covering a certain subject. DITA stores each topic in an individual XML file. The topics are further specialized into three types:

- Task
- Concept
- Reference

Task topics describe procedures using instructions in series of steps. Concept topics provide overviews, definitions and explanations. Reference topics contain detailed and specialized information in an ordered form.

5.6.3 AsciiDoc

AsciiDoc is a lightweight markup language, which is easily readable while using customizable plain text markup conventions. AsciiDoc files can be translated to HTML and DocBook, which can be further exported to final formats, such as PDF, EPUB or DVI. AsciiDoc defines three types of documents [2]:

- Article
- Book
- Manpage

Articles are used for short documents and generally for any documentation. Books include specific sections, e.g. preface. Manpages are used to generate manual pages in roff format.

5.6.4 Wiki

Wiki markup is a lightweight markup language used to create wiki pages. A wiki page is usually represented by a HTML code, which is generated from editable source code, and a page displayed by a browser from the HTML code. The editable source code uses simple syntax, which differs on different wiki systems.

The popularity of using wiki for creating software documentation has been increasing due to several reasons. The documents can be created by non-technically experienced users, it provides a comfortable way of collecting customer feedback and facilitates the maintenance of existing documentation (eliminates the need to export documents to presentable formats and upload them the servers). Many wiki systems also support conversions to standard publishing formats, such as PDF.

5.7 Documentation Software Tools

Apart from proprietary documentation tools and wikis, most tools are able to work with multiple source formats or at least they are capable of converting various formats into their native one. Therefore, the tools are listed in a separate chapter.

5.7.1 Adobe Technical Communication Suite

Adobe Technical Communication Suite is a collection of tools designed for creating technical documentation, the latest version being number 4. The most essential tools from this set are Adobe FrameMaker (latest version 11) and Adobe RoboHelp (latest version 10).

Adobe FrameMaker is an authoring and publishing tool designed to create complex documents. It provides WYSIWYG as well as XML code editors and support for DITA standards.

Adobe RoboHelp is a help authoring tool used for publishing documents on-line. It is capable of delivering output in WebHelp, CHM, PDF or eBook formats and is usually used in a combination with Adobe FrameMaker.

5.7.2 Help & Manual

Help & Manual is proprietary documentation tool developed by EC Software. The latest released version at the time of writing this thesis is version 6.4. It stores content in custom XML format of files, which can be edited using a WYSIWYG editor. The editor is intuitive to use providing a user interface similar to Microsoft Office, which makes this environment accessible to a vast range of users. One of its most useful features is the capability of publishing to various formats, including HTML Help, Webhelp, PDF or Microsoft Word.

Help & Manual has been used by Kentico for several years. Its disadvantages and reasons for transitioning to a different authoring system are described in section 6.2.

5.7.3 Confluence

Confluence is a team collaboration software developed by Atlassian, powerful Australian software company. It is a very popular wiki system, which apart from serving companies with a robust solution for organizing internal knowledge, also provides rich features for authoring technical documentation. In the beginning, Confluence used wiki markup language, but this support was discontinued with version 4.0 and replaced by autocomplete and autoformat functions, which automatically convert wiki markup to rich text (without the

option of converting it back). Although Confluence is mainly focused on direct on-line publishing, it can also export content to PDF, MS Word, HTML and other formats.

Confluence is currently offered under two types of licenses:

- OnDemand
- Downloaded

OnDemand version is based on a monthly subscription for hosted service in the cloud with little maintenance required from the buyer's side. The Downloaded version is based on one-time purchase and requires hosting on buyer's servers. However, this subscription allows for various customizations of content and enables the buyer to install additional features from the Attlassian Marketplace, which is not available with the OnDemand subscription.

6 Adaptation of Technical Writers for Agile Environment

At the time of writing this thesis, Kentico was in the process of transition to the agile project management using Scrum. The transition was not complete and perfect and some processes still needed to be improved. One of these imperfect processes was the documentation. The Technical Writers were still documenting new functionality of the product using the Waterfall style. When the functionality was implemented, the team informed the Technical Writer to start creating the documentation for the functionality. Though this practice worked in the past, it needed to be changed so that the Technical Writers could be integrated into the agile environment of the company.

The first chapter focuses on describing the character of the Kentico company and the historical events that led to the need of transforming the traditional documentation process to agile. The position of Technical Writers at the company is depicted as well, along with the problems of the documentation process.

The following chapters then continue with suggestions for adjusting the environment in which the Technical Writers could work in the agile way. These chapters also provide a foundation for the next chapter in which these suggestions are put in practice and tried out on two agile projects.

6.1 Situation in Kentico

Kentico had, at the time of writing this thesis, about 110 employees in the office located in Brno and about 20 employees in offices in other countries. The organizational structure of the company is rather flat and the company does not create unnecessary levels of management positions and puts more autonomy into its employees. Therefore, more than half of the total number of employees is concentrated in the development department of the company.

The company is developing one product, the content management system (CMS). The most of the customers of the company are

located in the USA, UK and Australia, which is why the product's user interface and documentation is written in English. The single product has been developed for 9 years with the last released version of number 7.0 and the version 8.0 currently in development. This means that there are approximately yearly releases of new versions of the product. In connection with the transition to the agile principles, the management of the company plans on dramatically shortening the release cycles. This is also the reason why the documentation process had to be adjusted, so that it could be adapted to shorter releases (whether it is three months or three weeks) and not to fall behind. With every new release, new functionality is added to the product, changes are made to the existing functionality of the product and some releases also include a visual change of the user interface which must be reflected in the documentation.

6.1.1 Development History

In the past, the planning of the product development was done in the traditional way according to the Waterfall software development life cycle. One whole release of the product was planned in advance at the beginning of the development. The new features planned for the release were selected by the CEO and CTO of the company and their time demands roughly estimated by individual teams based on previous experience. The development proceeded according to the plan for approximately a year. A new version of the product was released then, and the cycle began anew.

As the company grew, this approach no longer seemed viable. There was a need to adapt to changes in technologies during the development and to respond to the customer requirements quicker than in a year's time. This development cycle also put enormous pressure on the whole development department including the Technical Writers, before the end of the release period.

Therefore, the company decided to come over to the agile software development practices, particularly to Scrum.

6.1.2 Current Scrum in Kentico

Kentico has been in the process of integrating Scrum for almost two years. The change did not happen overnight, but was gradually introduced into the way the development worked.

At this point, it would only be appropriate to mention that, according to some authors ([21], [15]), there is no such thing as custom Scrum. A company either has to adopt all Scrum principles, roles and events, or the result is not Scrum. Even though it has not always been so, Kentico currently satisfies this rule despite certain imperfections in the processes.

There are 10 development teams, each of them centered around certain functionalities of the product (for example, the E-commerce team, the Core team, etc.). Each team has a Team Leader, who is typically the most skilled programmer in the team, two or three other developers, one or two testers, a UX designer (designer of the user interface with regard to easy usability) and a Technical Writer. Every two teams share a Scrum Master, so the Scrum Masters have enough time to steer the teams in the right direction, help them improve in what they are doing and solve their problems.

The customers are represented by a few Product Owners, whose responsibility is to learn the customers' needs, gain their feedback on the developed functionality and decide which features need to be implemented and which do not. They write down the requirements for the new functionality using user stories in the format described in section 5.5.1, but without the acceptance criteria. The following is an example of a typical user story:

- As a content editor, I want to be able to create an on-line form so that I do not have to ask the administrator to do it for me.

6.1.3 Technical Writers in Kentico

There are five Technical Writers in the company. One of them is a Technical Writer Coordinator, who takes care of organizational matters, resolves uncertainities and tries to find ways to improve the existing documentation processes. Each Technical Writer is assigned to one, two or three development teams and sits in the office of one of these teams.

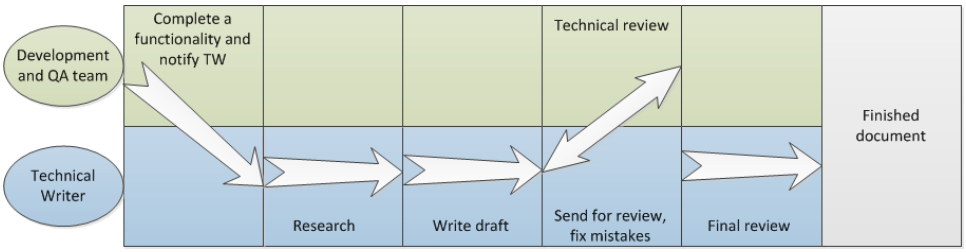


Figure 6.1: The documentation process in Kentico

The Technical Writers as a group do not have a team leader or a superior, who they would answer to. The Technical Writers belong to the teams and their supervisors are the Technical Leaders in the teams.

The documentation in the company plays an important role. This is caused by the fact that the developed product is extensive and very complex and requires a lot of configuration. Also, the company builds on a quality customer support, for which to achieve is important to provide correct and comprehensive documentation.

6.1.4 The Documentation Process

When the developers finish some functionality, they notify their assigned Technical Writer through a company's ticket system. The Technical Writer begins by trying out the functionality in the system, asking the developers about the functionality and deciding which information the customers would need. Then the Technical Writer writes the documentation down, creates screenshots of the user interface and draws diagrams if necessary. Subsequently, the documentation is reviewed by a member of the team that created the functionality. If a technical mistake in the documentation is found, the reviewer sends the documentation task back to the Technical Writer. After all mistakes are corrected and the reviewer does not find any other mistakes, the Technical Writer does a final review, correcting grammar and typographic mistakes. After that, the documentation of the feature is considered as done and can be uploaded to the company's website. The current documentation process in Kentico is illustrated on figure 6.1.

6.1.5 Specifics and Problems of the Documentation in Kentico

The problem with the documentation process described in the previous section is that, very often, the functionality is further modified after the documentation is finished. The appearance or text in the user interface is changed, the behavior of the functionality is adjusted or some new elements are further added. These additional changes usually mean that a part of the documentation has to be rewritten and screenshots recreated. What is the most problematic though, is for the Technical Writers to keep track of such changes. If the developers who made the modifications do not notify the Technical Writer, and the problem is that they might not realize which modifications affect the documentation, the modifications of the functionality are not often propagated to the documentation.

Another issue is the audience for the documentation. In many cases, the Technical Writers are not sure who they are writing for and who will eventually read their particular section of the documentation. The company is not closely acquainted with its customers and the Technical Writers are not often able to determine what the customers would need to read in the documentation to operate the system more easily and effectively. Moreover, parts of the developed product are used by different roles, for example the web developers who write custom code for parts of the system, website administrators who configure the environment and settings, and editors who add and edit the content of the created websites. In the future, this information about customers should be provided by Product Owners, however, these positions have been established only recently, and the Product Owners have not been able to gain that knowledge yet.

6.2 Used Documentation Tool

The software tool currently used for creating user documentation in Kentico is Help & Manual 6.1.0. Though this tools has served the Technical Writers well, several issues have been identified that would impede the process of creating user documentation in agile environment. Those identified issues are:

- Postponed technical reviews

- Complicated publishing
- Complicated releases and updates
- No built-in version control
- Low quality search functions
- Low quality user support [18]

When Technical Writers finished a documentation draft, they could not immediately send the document for a technical review by the development team. They had to wait until the documentation was automatically regenerated (once a day), because only the Technical Writers' user accounts had licenses for the Help & Manual and thus to the sources of documentation. This slowed down the review process.

This tool uses HTML page templates for generating HTML-based documents from the source files. These templates were difficult and time consuming to maintain.

When a new version of the product was released, the documentation for this version had to be generated and uploaded to the company's websites. This included voluminous PDF files of user guides, tutorials and specialized guides and their HTML versions, which comprised of hundreds of files. Moreover, the same upload procedure had to be done with every update of already published documents.

Version control was not a native part of the tool. Even though the tool provided support for external systems, this option was not applicable in the company.

Furthermore, a few features which would be appreciated in a new tool have been summed up by the Technical Writers in the company:

- Allow developers and QA engineers to update the documentation
- Provide a better contact with customers

User documentation in Kentico contains mainly information for developers working with the CMS. It contains, among others, code examples, reference pages on API methods and setting keys and similar specialized data. It would be beneficial, if the developers and QA

engineers could fix minor mistakes in these sections of the documentation during their reviews.

User documentation is supposed to help users in their work with the product. In order to provide the best service, collecting customer feedback continuously is essential. Not only is the customers' opinion about the quality of the documentation important for the Technical Writers as well as the Product Owners, but also their suggestions for uncovered topics, notices about outdated information and generally any comments are highly appreciated.

For these reasons, it was decided that a wiki system best covers the described company's and Technical Writers' needs. The system to replace Help & Manual was chosen to be Confluence, for its simplicity, strong background in the Atlassian company and possible integration with other useful Atlassian products, such as Jira, a tool for managing agile projects, or Greenhopper (newly renamed to Jira Agile), a tool for displaying Scrum boards.

A quick research among the company's partners showed that the main disadvantage of wiki based help systems, the necessity of constant access to the Internet, is not a problem due to the nature of work with the CMS through a web browser.

6.3 Team and Collaboration

Technical Writers already sit together with the development teams, but what the other team members need to understand, is that instead of a service, they are now part of the Scrum Team. Technical Writers will participate more on the creation of the product. They will attend all the Scrum meetings, provide feedback on the functionality from the users' perspective, choose the best text for the user interface and act as additional level of testing.

This can be a problem if a Technical Writer is part of more than one team. In such case, and this is also the case of Kentico, the Technical Writer sits with the team that has greater demands on documentation and chooses which Scrum meetings to attend so as to utilize the work time effectively. The Technical Writer also needs to make use of his or her skills to communicate properly with all teams [9].

The question is though, whether to include the documentation

in the Definition of done. Each team in Kentico is different and may have different priorities, and also the Product Owners may not value the documentation in every projects the same. Therefore it was decided, not to generally apply this principle. Every team will decide on its own.

Additionally, the template for writing user stories had to be changed to the one described in section 5.5.1. This meant that the Product Owner collaborates with the team on writing the acceptance criteria with every story.

6.4 Adapting the Documentation Process

In agile environment, the Product Owner enters the documentation process and plays an important part in it, as illustrated on figure 6.2. At first, the Technical Writer together with the Product Owner identify user stories that will require creation or update of the documentation. Then the Technical Writer or Product Owner creates documentation sub-tasks for identified user stories and specifies details and demands on the documentation. During the Sprint planning meeting, the Technical Writer roughly estimates the time needed to complete the documentation for user stories chosen for the Sprint, based on previous experience. The Technical Writer states whether it is possible to finish the documentation within the Sprint. If it is not possible, the Team has to find a way to help the Technical Writer with the documentation or refuse some user stories.

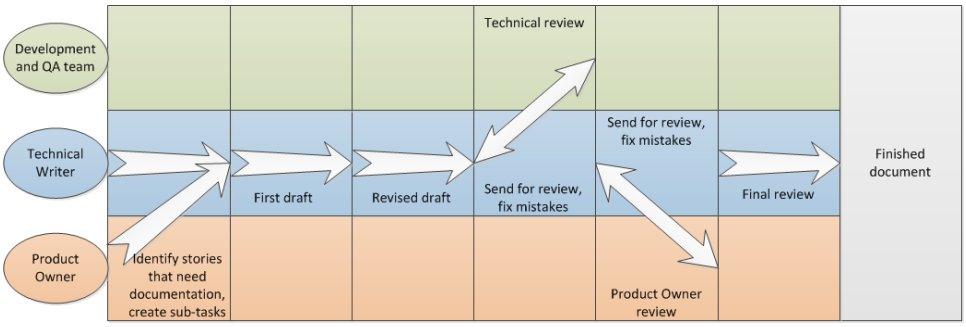


Figure 6.2: Agile documentation process

When the Sprint starts, the Technical Writer writes documentation drafts for the user stories based on the information and acceptance tests provided in the description of the user stories. Technical Writer reviews the draft and tests it against the developed functionality. When an adequate draft has been created, the Technical Writer submits the draft for review by the other members of the team, who check the technical accuracy of the documents. When technical mistakes have been corrected, the Technical Writer submits the documentation for review by the Product Owner, who inspects the business value of the documentation. The documentation is finished when it does not contain any technical, grammatical or typographical mistakes and it adds appropriate business value approved by the Product Owner.

7 Case Study

This chapter describes my own experience while working in Kentico as a Technical Writer. Two projects in two separate agile teams were chosen to be presented in this thesis. Both teams adopted Scrum practices and tried hard to follow agile philosophy and principles. The work in these two teams was my first attempt of agile documenting and gave me valuable experience and insight.

Both projects and teams were fundamentally different, so it would not have been sufficient to present only one of them in this thesis. Each project had its specific issues and difficulties the teams had to face.

This experience was chosen to be described by the course of Scrum meetings. This approach maps the meetings that are essential for the Technical Writers, since the time spent on Scrum meetings reduces the time spent on the documentation and has to be divided with care. This approach also reveals problems with the suggested documentation process and integration of Technical Writers into the agile development.

7.1 Team A

The team A consists of 6 people in total. There is one Technical Leader, two developers, one tester, UX Designer and a Technical Writer. The team also has a dedicated Scrum Master.

This team develops new modules for the CMS and takes care of selected existing modules. Like the majority of other teams, this team also gets its work assigned by the management of the company, which was also the case of this project. Specifically, this project was bestowed on the team by the Chief Technology Officer (CTO), who also acted as the Product Owner (PO) during the project development. This conflict of roles (CTO and PO) in one person, however, defies the Scrum practices and proved to be ineffective and hindering on many Scrum meetings.

The developers, tester and UX Designer sit together in one office. The Technical Writer though, is shared between two teams (say, teams A and C) and sits in another office. This was thought to bring

problems with communication and availability during sprints, but turned out not to be an issue.

7.1.1 Planning the Project

The Technical Writer did not participate in the planning of this project, but it is still relevant to explain the conditions that preceded the development phase.

At first, the Product Owner got an idea of what the planned feature should do. In fact, it was not a new feature, but rather a new user interface for an already existing functionality. The purpose of this new interface was to allow non-technically skilled end users, for example marketers, to easily create a new on-line form. The Product Owner created a first draft of the project backlog using user stories, which represented individual features of the new interface.

Then the UX designers created a visual mock-up of the new interface, as can be seen on figure 7.1. This unified the terms used for individual components of the interface and allowed everyone to visualize the final result.

The team then roughly estimated the time needed to complete user stories in the project backlog. The Product owner prioritized the backlog and divided it into two parts. The first part was named the Minimal Marketable Product, i.e. the minimal part of the functionality which would bring the customers a measurable value. In the second part were user stories that brought the customers additional "nice-to-have" features, which should be completed only after the team finishes the Minimal Marketable Product and only if there is enough time at the end of the project.

According to the estimates in the Minimal Marketable Product, the team's velocity and other interests of the company, the time box for this project was established to three months.

The documentation for this project was supposed to consist of the help for non-technically skilled users in the form of a list of tasks.

7.1.2 Backlog Grooming

During the work on the project, the Backlog grooming meetings were being planned as necessary. The Product Owner specified the details

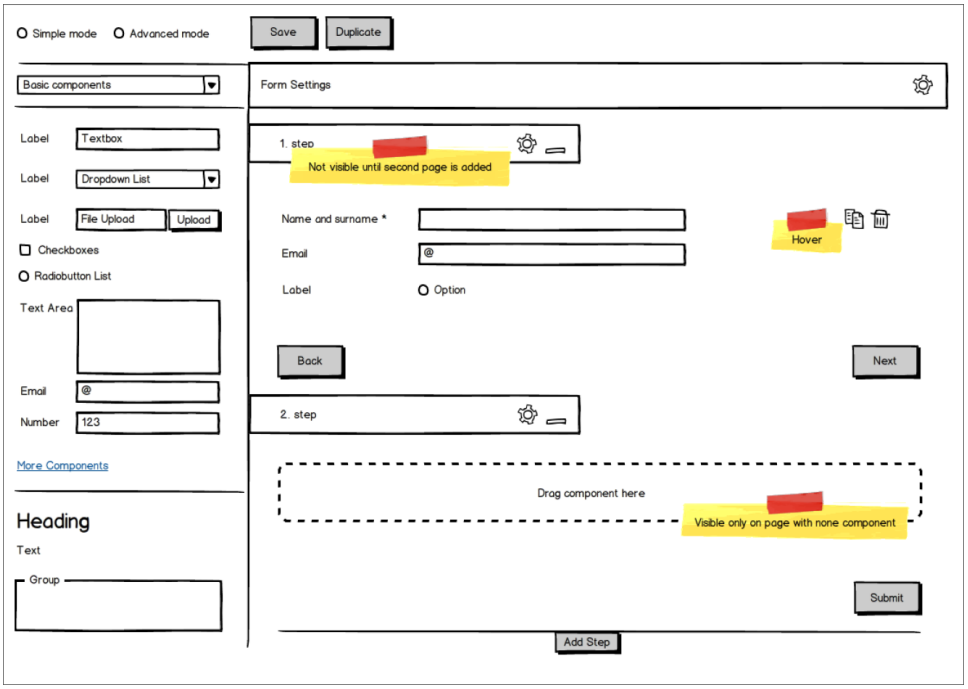


Figure 7.1: Mock-up of the planned functionality

of individual user stories and consulted them with the team. After the Product Owner and the team agreed on the details, the Product Owner wrote the details down using a user role and the objective the user wanted to achieve. Subsequently, the Product Owner together with the Team wrote down the acceptance criteria in the form of test cases, whose accomplishment marked the task as completed.

The following is an example of one of the first user stories in the project:

- As a content editor I want to arrange the order of the fields in a preview of an existing form.
  - Using a WYSIWYG editor
  - Drag & Drop principle
  - Basic components only
- Acceptance criteria:
  1. Open an existing form.
  2. Change the order of 3 fields.
  3. Verify on the live site that the order of the fields has been changed.

Attending Backlog grooming meetings and seeing user stories defined in this level of detail was beneficial for the Technical Writer. Written descriptions provided enough information to determine, which stories needed documentation and which did not. For example, arranging order of fields did not need documentation, as it was intuitive enough for the defined user role. It was also possible to create a first draft of the help task for the particular user story before the functionality was even developed. This was facilitated by using the details in the story with the combination of visual mock-up. All the necessary information was provided from the beginning of the project – the target user role of the documentation, what the user role wanted to achieve, how the goal is achieved and which terminology is to be used to keep the documentation consistent with the interface.

7.1.3 Sprint Planning Meetings

On the Sprint Planning meetings, the Product Owner presented the top user stories from the Project backlog to the team. As the user stories already contained details from the Backlog grooming meetings, the team only asked the Product Owner about points they did not understand clearly. Then the Product Owner left the meeting and the team estimated how many user stories will they commit to complete in the Sprint according to the team's velocity and availability. Then the team broke up the user stories to individual sub-tasks.

The Technical Writer identified user stories, which would need documenting and created documentation sub-tasks for them. According to her availability during the upcoming Sprint, the Technical Writer then assessed, if she has enough time to create or update the documentation. In case she did not have enough time for the documentation, the team would have to help her or some user stories would have to be omitted. This case though did not occur in this project.

The benefits of attending these meetings were that the Technical Writer could easily track the progress of developed functionality and knew what to document.

However, the benefits often did not pay off for the time spent on these meetings. The problem of this team and this project was that the modules owned by the team had a severe technological debt and generated large amounts of bugs and defects. Even though the team focused on fixing only the most severe bugs, this additional work significantly lowered the time the team could spend on the development of the functionality. Therefore, the progress was slow and generally there was only one task to document in the whole two-week Sprint. Since the functionality was designed as intuitive as possible and therefore did not require much documentation, one such task would take only about two hours to complete. Moreover, several Sprints did not contain any documentation tasks at all.

Based on these reasons and after considering that the Technical Writer could spend her working time better, she stopped attending these meetings. The only action the Technical Writer had to do after this decision was to check the Sprint backlog after the planning ended, identify the user stories which needed documentation in the Sprint and create documentation sub-tasks for these user stories.

7.1.4 Daily Meetings

On the daily meetings, the team gathered and each member of the team explained what they have done since the last meeting, what will they do and which impediments they had to face. Due to the facts described in the previous section, the Technical Writer often did not have any work to report to the team. Despite that, she attended the daily meetings at least once a week just to keep in touch with the team. This proved useful, as it provided an opportunity for the members of the team to ask for grammar checks of the resource strings or to assign the Technical Writer some lesser unplanned tasks (for example, to provide tooltips for the elements of the interface).

7.1.5 Sprint Reviews

On the Sprint Review meetings, one member of the team presented the completed user stories to the Product Owner in the form of visual inspection of the developed functionality. The first few Sprints did not include any documentation to present to the PO and it later became a practice to show only the functionality and not the documentation that goes with it. The documentation was presented to the PO somewhere in the middle of the project and fortunately, the PO approved it. If he did not though, it would have been necessary to rewrite the work done up to that time. A better solution would have been to present the documentation to the PO on every sprint review, so that he could express his comments.

The benefit of attending these meetings was that the Technical Writer could verify her work against the completed functionality in the sprint. The disadvantages were that the Product Owner often slipped into his CTO role and wanted to inspect the technical details of the solution and the code. Again, the time spent on these meetings proved to be inefficient. This problem was caused by an imperfect implementation of the Scrum methodology. The team should have had a dedicated Product Owner without the conflict of more roles upon him.

After deciding not to attend these meetings, the Technical Writer only needed to inspect the delivered functionality on a newly built version of the CMS to gain the same level of knowledge.

7.1.6 Sprint Retrospectives

On the sprint retrospective meetings, the team members identified problems that arose during the Sprint and acknowledged what went well. Due to the fact that the Technical Writer was always able to finish the required documentation tasks in time and that she had plenty of time for ad-hoc work requested by the team, the documentation was rarely the subject of the retrospective meetings. Also, from the Technical Writer's point of view, the team was helpful and answered all asked questions quickly either face-to-face or through on-line communication. Therefore, there was no need to attend these meetings. Instead, the Technical Writer asked the team to contact her directly if anything was amiss.

7.1.7 Backlog Grooming

Even though there were some changes in the design of the implemented functionality during its development, the implementation was not too different from the original mock-up. The amount of features eventually implemented was a little larger than the Minimal Marketable Product, so not all user stories from the Project backlog and the mock-up were brought into reality, as can be seen by comparing figures 7.1 and 7.2.

As was already mentioned, the interface was highly intuitive, so in the end, only a few user stories needed documentation. An excerpt of the documentation for this project created in Confluence can be seen on figure 7.3.

7.1.8 Summary

After a few initial Sprints, the Technical Writer stopped attending the Scrum meetings with this team except for the Backlog grooming and occasional Daily meetings and Sprint reviews. The specific aspect of this agile project was that there was not enough work to keep the Technical Writer occupied all the time during Sprints. Because the second team assigned to the Technical Writer required even less documentation work, the Technical Writer had to pick up documentation projects from other teams or look for technical debts and rewrite

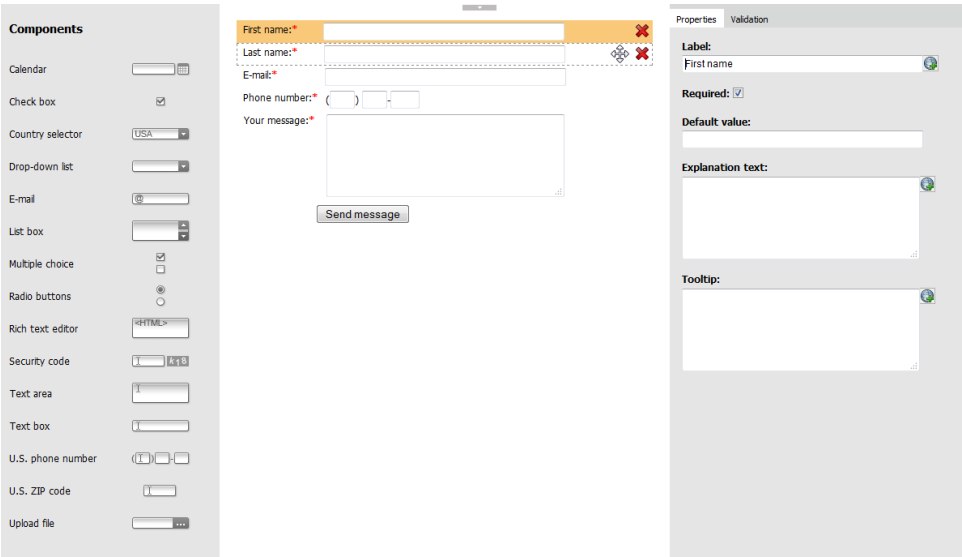
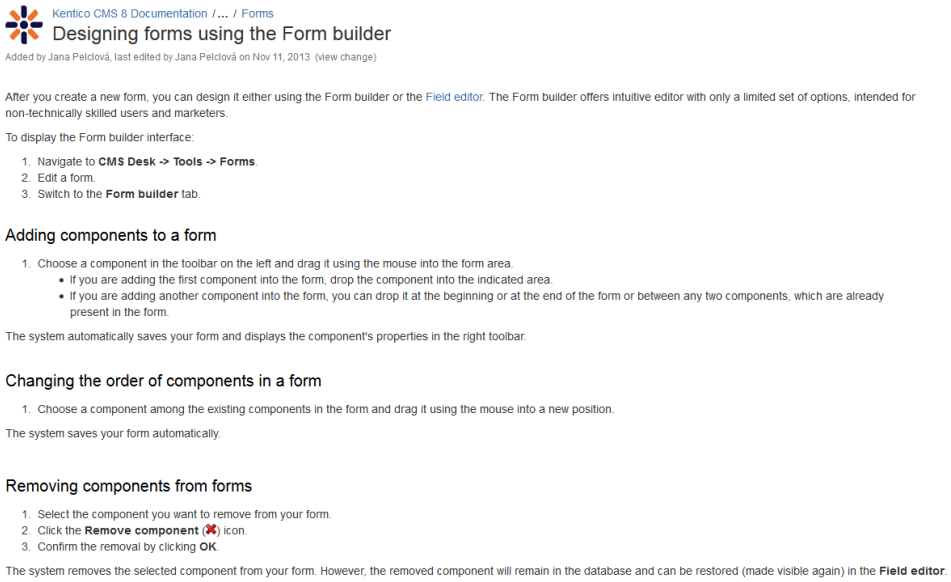


Figure 7.2: The resulting functionality of project A



Kentico CMS 8 Documentation / ... / Forms

Designing forms using the Form builder

Added by Jana Pelclová, last edited by Jana Pelclová on Nov 11, 2013 (view change)

After you create a new form, you can design it either using the Form builder or the Field editor. The Form builder offers intuitive editor with only a limited set of options, intended for non-technically skilled users and marketers.

To display the Form builder interface:

1. Navigate to **CMS Desk -> Tools -> Forms**.
2. Edit a form.
3. Switch to the **Form builder** tab.

Adding components to a form

1. Choose a component in the toolbar on the left and drag it using the mouse into the form area.
   - If you are adding the first component into the form, drop the component into the indicated area.
   - If you are adding another component into the form, you can drop it at the beginning or at the end of the form or between any two components, which are already present in the form.

The system automatically saves your form and displays the component's properties in the right toolbar.

Changing the order of components in a form

1. Choose a component among the existing components in the form and drag it using the mouse into a new position.

The system saves your form automatically.

Removing components from forms

1. Select the component you want to remove from your form.
2. Click the **Remove component** (✖) icon.
3. Confirm the removal by clicking **OK**.

The system removes the selected component from your form. However, the removed component will remain in the database and can be restored (made visible again) in the **Field editor**.

Figure 7.3: Excerpt of a documentation page created for the project of team A in Confluence.

outdated sections of the documentation.

Keeping the agile attitude in this project enabled the Technical Writer to keep track of all the changes in the functionality and adapt the documentation accordingly. Even though she had to subsequently rewrite some parts of the documentation which were already written in previous Sprints, this approach helped the Technical Writer adapt to the changes quickly and keep the documentation up-to-date at the end of each sprint.

The Technical Writer did not have any problems writing the help tasks in this project, because she had plenty of time during Sprints and was able to test the developed functionality herself. Moreover, as this functionality was aimed at the end users of the CMS, the Technical Writer could easily empathize with them and determine what they needed to know, even better than the more technically oriented Product Owner.

After the experience gained in this project the Technical Writers from the company decided to create a documentation backlog, which would contain tasks that are not subjects to Sprints. These tasks could represent, for example, maintenance of the documentation topic structure, updates of code examples, rewriting of the obsolete feature-based documentation to task-based approach and work on long-term projects or guides. The Technical Writers will be able to choose one of these tasks whenever their time is not fully utilized by the demands of Sprints.

Furthermore, to better utilize the Technical Writer's time, she was assigned to the more demanding team B in exchange for team A. This experience is described in the following chapter.

7.2 Team B

The team B consists of 8 people in total. There are five developers including the Technical Leader, one tester, a UX Designer and a Technical Writer. The team also has an assigned Scrum Master.

The team's B responsibilities are building cloud based support for Kentico CMS and developing new security features for the CMS. The Product Owner for this team is solely focused on overseeing the development of new features for the CMS and communicating with

the customers.

This team is highly effective and regularly produces a lot of functionality. It has already adopted all Scrum practices and continuously tries to improve the processes within the team.

All members of this team sit in one office except for the Technical Writer, who is sitting in another office with a team C. However, such condition resulted in problems this time, which will be described later in this chapter.

7.2.1 Planning the Project

The project described in this chapter was planned only for one two-week Sprint. Its purpose was to develop a support for a new authentication method in the CMS. This was a completely new functionality which had to be programmed and integrated into the CMS. The user interface for this functionality was supposed to be a new page with a group of settings, which the administrators would have to configure to enable the functionality. Therefore, there was no need to prepare a visual mock-up this time.

The Product Owner prepared a backlog with user stories already in the preferred format using a user role and acceptance criteria. The Product Owner marked a group of user stories as the Minimal Marketable Product and the rest were advanced features, which would be nice to have if the team had enough time for them.

The documentation for this project was aimed at administrators, who will be configuring and enabling the functionality in the CMS. This was all the information available about the documentation before the beginning of the project, because there was no visual mock-up to provide guides.

This project was planned for only one sprint and thus did not need backlog grooming meetings – all user stories were specified at the beginning of the project and it was not advisable to add new user stories to the project during its course.

7.2.2 Sprint Planning Meeting

On the single Sprint planning meeting of the project, the Product Owner presented the user stories to the team and they discussed the

details. At this point, the Technical Writer could not determine what documentation would be required for this project. It was known, which user roles was the planned feature aimed at, but the Technical Writer was not familiar with the technology and could not designate, which information the user role would need. In this case, the Product Owner was familiar with the target user role better than the Technical Writer and was able to specify the type of information that was to be provided for the user role in each user story.

When the team gained all the necessary details about the user stories, they estimated the amount of work required to satisfy the user stories. The Technical Writer though, was not able to estimate the time needed to write the documentation, as she did not have experience with similar projects and could not assess the scope of the documentation. Moreover, the documentation was to be created in two weeks of time, which seemed at that time as enough.

The team then accepted the Minimal Marketable Product and a few advanced user stories into the sprint. This meant that the documentation for this project would consist of a page with an introduction and description of the particular authentication method, instructions on how to configure the required settings and a code example, which would illustrate the possibilities of implementing an advanced custom functionality.

At the end of the meeting, the team broke up the use stories to individual implementation sub-tasks. Creating the documentation sub-tasks was the responsibility of the Product Owner.

7.2.3 Daily Meetings

On the Daily meetings, the members of the team reported the information as specified in section 4.4.2. Despite the fact that the team consisted of 8 people, the Daily meetings went swiftly and clearly, as everyone had prepared what to report.

The daily meetings with this team were essential. Even though the Technical Writer could monitor the development progress of the Sprint in an internal project planning tool, these meetings provided ground for synchronizing with the team, asking question (after or before the meetings), and generally, keeping in touch with the team.

7.2.4 Sprint Review

On the Sprint review meeting, one previously selected member of the team presented the work completed in the sprint to the Product Owner. The team member went through each user story, presented the achieved solution and displayed the documentation. The Product Owner assessed the solution and the documentation and provided feedback.

In this Sprint, the team managed to develop all the functionality planned for the sprint. The documentation, however, was barely finished and most of the documentation sub-tasks were in the "review" state. Despite this fact, the team considered the Sprint as completed, because they were convinced that implementing and testing the functionality was enough.

The Product Owner was not content with this outcome though. He demanded completing the documentation tasks as quickly as possible and claimed that such result of a Sprint would be unacceptable in the future.

Based on this experience, the team added the "Completed documentation" condition into their Definition of Done, agreed that the documentation is part of the delivered increment of the product, and that the whole team should be concerned about its completion.

7.2.5 Sprint Retrospective

Since the documentation was the only work that was not finished during the Sprint, the team agreed to devote the time of the Sprint retrospective meeting to analyzing this issue. Not that there were not any other problems to discuss, but the team itself identified this particular issue as the most pressing.

Through a discussion were identified four main difficulties that hindered the documentation process which resulted in a failure to finish the documentation in time. The difficulties were written on a whiteboard and the team proposed solutions which would help avoid these problems in the future. The chosen solutions were written on the board using a green marker, as can be seen on a figure 7.4.

Identified problems:

1. Late reviews
2. Lack of communication
3. New information does not reach the Technical Writer
4. Unsuitable code examples

After a documentation task was processed, it was sent to the "Review" state so that one of the team members could check the technical correctness of the information. The team was always busy though, dealing with implementation and testing problems, and did not give high priority to the documentation review. But when one team member found mistakes in the documentation and the Technical Writer corrected them, another team member found different mistakes during his review of the corrections. Furthermore, review by the Product Owner was done after the technical review (according to the documentation process suggested in section 6.4) and revealed that important information about the feature is missing. This would not have been a problem, if the feedback came early enough and not near the end of the Sprint. The solution the team has agreed on, was that whenever the Technical Writer finishes a task, she will notify the team through an on-line communication and urge the team to do the review as soon as possible. Also it would be better if the review by the Product Owner was done before the technical review. This would ensure that mistakes in the documentation are found early enough to allow the Technical Writer to correct them before the Sprint ends.

This type of project did not allow the Technical Writer to explore the functionality directly in the developed program. She had to rely on materials available on-line and inquire other team members about the functionality. This was very difficult due to the fact that the Technical Writer was not sitting with them in one office. The only choices to communicate with the team were through on-line messages or by visiting them personally. Communicating on-line was not very effective, as the response times of the people varied, it took a long time to convey a complex thought and sometimes, redirections happened because the contacted team member did not know answers to the questions. Visiting the office provided better results, but doing so five times a day was not effective as well. One solution seemed to be

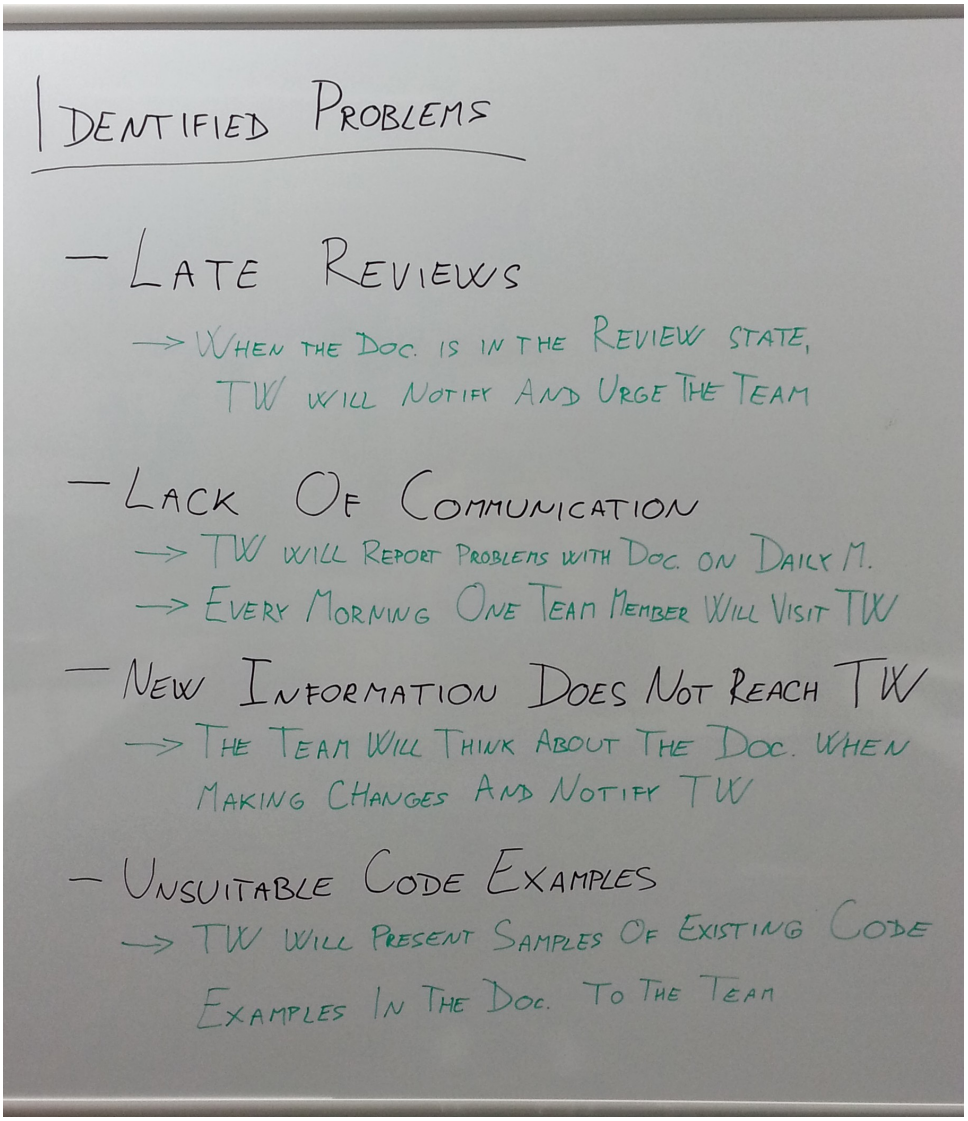


Figure 7.4: Identified issues in the documentation process on the sprint retrospective and proposed solutions

the relocation of the Technical Writer to the office of team B. However, this actually would not have solved the problem in general, as there were twice as many teams as Technical Writers in the company. The team proposed that one team member would visit the Technical Writer every morning and answer questions and thus provide a link between the team and the Technical Writer. Lack of communication also caused that the team was not informed that the documentation would not be finished before the end of the Sprint. The Scrum Master proposed that the Technical Writer reports all problems with the documentation on daily meetings.

When the team decided to make changes during the development (for example, to change the terminology or technologies used in the project), they did not inform the Technical Writer about them. To avoid this trouble, the team would consider if the intended changes would affect the documentation and inform the Technical Writer in the future.

The Product Owner wanted the documentation to provide an example of how to implement a custom functionality. So the developers extracted a part of their test code and sent it to the Technical Writer to serve as a simple example. However, the provided example was too simple and thus unusable for the documentation. Moreover, it was written in a different notation than the rest of the documentation examples, which took the Technical Writer a lot of time to convert to the preferred format. As a solution to this difficulty, the team agreed to get acquainted with the format of code examples presented in the documentation to speed up the process next time.

7.2.6 Summary

This project in team B proved that communication and close collaboration is essential in highly productive teams. The Technical Writer is in a difficult position when trying to gain enough knowledge about the subject, and the other team members have to be available and willing to provide answers to questions. Also, the Product Owner has to understand the needs of target users of the functionality to determine the demands of the documentation.

This experience also showed that it is necessary to include the documentation into the Definition of Done of this team and consider

the documentation a part of the whole delivered functionality and the responsibility of the whole team. Completing the documentation within the Sprint also enables the Technical Writer to regard the work on the project as finished and move to a completely new topic once a following project starts.

7.3 Summary of the Case Study

Experience gained in these two projects helped Technical Writers and other roles in the company adjust their processes and it also improved the overall functioning of the Scrum methodology in the company. The lessons learned in these projects and actions taken after them are captured in the following list:

- Product Owners need to have only one role in the company.
- Product Owner helps define whether the documentation is included in the Team's Definition of Done.
- Review by the Product Owner comes before the technical review.
- The team regards the documentation reviews with high priority.
- Documentation is presented on Sprint review meetings along with the developed functionality.
- Technical Writers have to carefully choose between the benefits of attending Scrum meetings and the time spent on creating the documentation.
- A team member visits Technical Writer who is not sitting in the same office with the team.
- A documentation backlog was created.

8 Conclusion

The purpose of the thesis was to study agile principles including the Scrum methodology, learn about approaches to software documentation and to devise a way of creation and maintenance of software documentation in agile environment.

Based on the research done in the first five chapters, several suggestions were offered in chapter 6. The disadvantages of the documentation tool used back then were mapped and a new modern documentation platform in the form of a Confluence wiki was chosen according to the collected requirements. Various observations about the integration of Technical Writers into the Scrum teams have been made and the documentation creation and maintenance process has been revised to fit in the agile development.

The two case studies presented the experience gained in two different development projects from a Technical Writer's point of view. The suggestions from the previous chapter were tried out on real projects and their impact on the resulting user documentation was assessed at the end of each of the two studies. They proved that creating quality technical documentation in agile environment is possible and that it provides benefits to the Technical Writers as well as the company.

Additional adjustments, which were done based on problems identified through the case studies, helped the company further improved the implementation of the Scrum methodology and can serve other companies and Scrum teams in a similar manner.

Further work in this field could be focused on user stories and on improving the way they are specified so that the information included in them can further help the Technical Writers in their work.

Bibliography

[1] Agile Alliance: *The Agile Manifesto*. http://www.agilealliance.org/the-alliance/the-agile-manifesto/ [Accessed: 12/2013].

[2] *AsciiDoc User Guide*. http://asciidoc.org/asciidoc.pdf [Accessed: 12/2013].

[3] Austin, G. and Berry, M. *A Writer's Guide to Surviving Agile Software Development*. http://media.developerforce.com.s3.amazonaws.com/techpubs/agile_writers_guide.pdf [Accessed: 9/2013].

[4] Beck, K. *Extreme Programming Explained: Embrace Change*. Addison Wesley 1999. ISBN 0201616416.

[5] Beck, K.; et al. *Manifesto for Agile Software Development*. http://agilemanifesto.org/iso/en/ [Accessed: 12/2013].

[6] Bilheimer, S. *How to Become a Technical Writer*. Booklocker.com 2002. ISBN 9780970196415.

[7] Bureau of Labor Statistics, U.S. Department of Labor: *Occupational Outlook Handbook, 2012-13 Edition, Technical Writers*. http://www.bls.gov/ooh/media-and-communication/technical-writers.htm [Accessed: 12/2013].

[8] Cohn, M. *Succeeding with Agile*. Addison Wesley 2010. ISBN 9780321579362.

[9] Fox, A. and Meredith, K. *Mobile and Agile: The Floating Writer's Survival Kit*. http://www.writersua.com/articles/AGILE/index.html [Accessed: 9/2013].

[10] Gluckenheimer, S and Perez, J. J. *Efektivní softwarové projekty*. Zoner Press 2007. ISBN 9788086815626.

[11] Guren, L. *Going Beyond Tasks: Creating a workflow path*. Presentation at UA Europe Annual Conference, Manchester 2013.

[12] Hargis, G.; et al. *Developing quality technical information: a handbook for writers and editors*. IBM press 2004. ISBN 0131477498.

[13] Highsmith, J. *Agile Project Management: Creating Innovative Products*. Addison Wesley 2009. ISBN 9780321658395.

[14] Kosek, J. *Finální řešení pro vaši dokumentaci*. docbook.cz/clanky/uvod.html [Accessed: 12/2013].

[15] Lacey, M. *The Scrum Field Guide*. Addison Wesley Professional 2012. ISBN 0321554159.

[16] Nazzaro, W. and Suscheck, Ch. *New to User Stories?*. http://www.scrumalliance.org/community/articles/2010/april/new-to-user-stories [Accessed: 9/2013].

[17] Ošlejšek, R. *Objektové metody návrhu informačních systémů*. Faculty of Informatics MU, Brno 2012. https://is.muni.cz/auth/el/1433/jaro2012/PA103/um/prez/01_Strategie-vyvoje-sw.pdf [Accessed: 11/2013].

[18] Pěnička, P. *Software Documentation as a Service*. Master's thesis, Faculty of Informatics MU, Brno 2013.

[19] Rubin, K. S. *Essential Scrum: Practical Guide to the Most Popular Agile Process*. Addison Wesley 2012. ISBN 9780137043293.

[20] Rüping, A. *Agile Documentation: A Pattern Guide to Producing Lightweight Documents for Software Projects*. John Wiley & Sons 2003. ISBN 0-470-85617-3.

[21] Schwaber, K and Sutherland J. *The Scrum Guide*. https://www.scrum.org/Portals/0/Documents/Scrum%20Guides/Scrum_Guide.pdf [Accessed: 12/2013].

[22] Schwaber, K. *Agile Project Management with Scrum*. Microsoft Press 2004. ISBN 073561993.

[23] Shore, J. *The Art of Agile Development*. O'Reilly Media 2007. ISBN 0596527675.

[24] Sommerwille, I. *Chapter 30: Documentation. A revised chapter from the book Software engineering, 4th edition*. http://ifs.host.cs.st-andrews.ac.uk/Books/SE9/WebChapters/PDF/Ch_30%20Documentation.pdf [Accessed: 12/2013].

[25] Sutherland, J. *Agile Principles and Values*. http://msdn.microsoft.com/en-us/library/dd997578.aspx [Accessed: 12/2013].

[26] Wikipedie. *Publikování z jednoho zdroje*. http://cs.wikipedia.org/wiki/Publikov%C3%A1n%C3%AD_z_jednoho_zdroje [Accessed: 12/2013].